

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# **01**)

Apply modern simulation tools Modelsim and Logisim to implement basic processor components

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B__

Date performed: _19thAug _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task we write the Verilog code for seven basic gates (AND, NOT, OR, NAND, NOR, XOR, XNOR) by using Gate level modelling. The syntax of this model is followed in the following basic gates. The testbench is made for testing and it is same for all gates and there is a delay of #50 to see each combination in the wave. After compiling the code, the simulation is run then the wave is selected and it is formed on graph. The wave is in green color so there is no issue in the connection of wires.

Code:

AND GATE:

```
module logicgates(a,b,c);

input a;

input b;

output c;

and a1(c,a,b);

endmodule

module testbench();

reg x;

reg y;

wire z;

logicgates unit(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

end
```

```
endmodule
```

OR GATE:

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;
```

```
or a1(c,a,b);
```

```
endmodule
```

```
module testbench();
```

```
reg x;
```

```
reg y;
```

```
wire z;
```

```
logicgates unit(x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

NOT GATE:

```
module logicgates(a,c);
```

```
input a;
```

```
output c;
```

```
not a1(c,a);
```

```
endmodule

module testbench();

reg x;

wire z;

logicgates unit(x,z);

initial

begin

x=0;

#50 x=1;

end

endmodule
```

NAND GATE:

```
module logicgates(a,b,c);

input a;

input b;

output c;

nand a1(c,a,b);

endmodule

module testbench();

reg x;

reg y;

wire z;

logicgates unit(x,y,z);

initial

begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

NOR GATE:

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;
```

```
nor a1(c,a,b);
```

```
endmodule
```

```
module testbench();
```

```
reg x;
```

```
reg y;
```

```
wire z;
```

```
logicgates unit(x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

XOR GATE:

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;
```

```
xor a1(c,a,b);
```

```
endmodule
```

```
module testbench();
```

```
reg x;
```

```
reg y;
```

```
wire z;
```

```
logicgates unit(x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

XNOR GATE:

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;  
  
xnor a1(c,a,b);  
  
endmodule  
  
module testbench();  
  
reg x;  
  
reg y;  
  
wire z;  
  
logicgates unit(x,y,z);  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50 x=0; y=1;  
  
#50 x=1; y=0;  
  
#50 x=1; y=1;  
  
end  
  
endmodule
```

Output:

AND GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
y	1	Regi...	Internal	1
z	St1	Net	Internal	St1

OR GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
y	1	Regi...	Internal	1
z	St1	Net	Internal	St1

NOT GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
z	St0	Net	Internal	St0

NAND GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
y	1	Regi...	Internal	1
z	St0	Net	Internal	St0

NOR GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
y	1	Regi...	Internal	1
z	St0	Net	Internal	St0

XOR GATE:

Name	Value	Kind	Mode	Msgs
x	1	Regi...	Internal	1
y	1	Regi...	Internal	1
z	St0	Net	Internal	St0

XNOR GATE:

Name	Value	Kind	Mode	Msgs
x	0	Regi...	Internal	0
y	0	Regi...	Internal	0
z	St1	Net	Internal	St1

Task 2:

Description:

In this task the requirement is of writing Verilog code by using dataflow level modelling in which the syntax of writing the code is according to the model we used assign in it for each basic gate. Hence the test bench is also same for this modelling. Only the format of performing the operations is different as compared to gate level modelling. After that we run the simulation by selecting the signals and the graph is formed.

Code:

AND GATE (data flow):

```
module logicgates(a,b,c);
```

```
input a;

input b;

output c;

assign c= a&b ;

endmodule

module testbench();

reg x;

reg y;

wire z;

logicgates unit(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

end

endmodule
```

OR GATE (data flow):

```
module logicgates(a,b,c);

input a;

input b;

output c;

assign c= a|b ;

endmodule
```

```
module testbench();  
  
reg x;  
  
reg y;  
  
wire z;  
  
logicgates unit(x,y,z);  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50 x=0; y=1;  
  
#50 x=1; y=0;  
  
#50 x=1; y=1;  
  
end  
  
endmodule
```

NOT GATE (data flow):

```
module logicgates(a,c);  
  
input a;  
  
output c;  
  
assign c= ~a ;  
  
endmodule  
  
module testbench();  
  
reg x;  
  
wire z;  
  
logicgates unit(x,z);  
  
initial  
  
begin
```

```
x=0;

#50 x=1;

end

endmodule
```

NAND GATE (data flow):

```
module logicgates(a,b,c);

input a;

input b;

output c;

assign c= ~(a&b) ;

endmodule

module testbench();

reg x;

reg y;

wire z;

logicgates unit(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

end

endmodule
```

NOR GATE (data flow):

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;
```

```
assign c= ~(a| b) ;
```

```
endmodule
```

```
module testbench();
```

```
reg x;
```

```
reg y;
```

```
wire z;
```

```
logicgates unit(x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

XOR GATE (data flow):

```
module logicgates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;
```

```
assign c= a^b ;
```

```
endmodule

module testbench();

reg x;

reg y;

wire z;

logicgates unit(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

end

endmodule
```

XNOR GATE (data flow):

```
module logicgates(a,b,c);

input a;

input b;

output c;

assign c= a~^b ;

endmodule

module testbench();

reg x;

reg y;

wire z;
```

```
logicgates unit(x,y,z);
```

```
initial
```

```
begin
```

```
x=0; y=0;
```

```
#50 x=0; y=1;
```

```
#50 x=1; y=0;
```

```
#50 x=1; y=1;
```

```
end
```

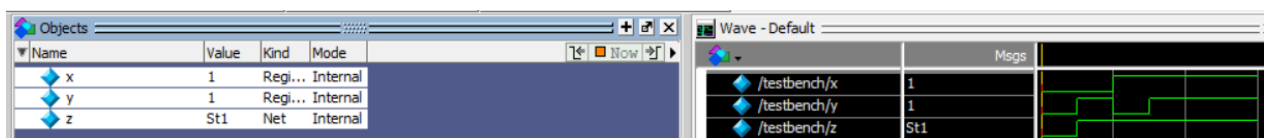
```
endmodule
```

Output:

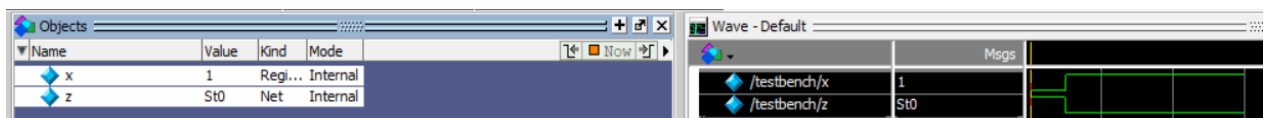
AND GATE (data flow):



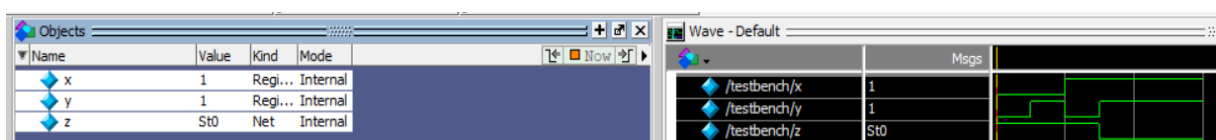
OR GATE (data flow):



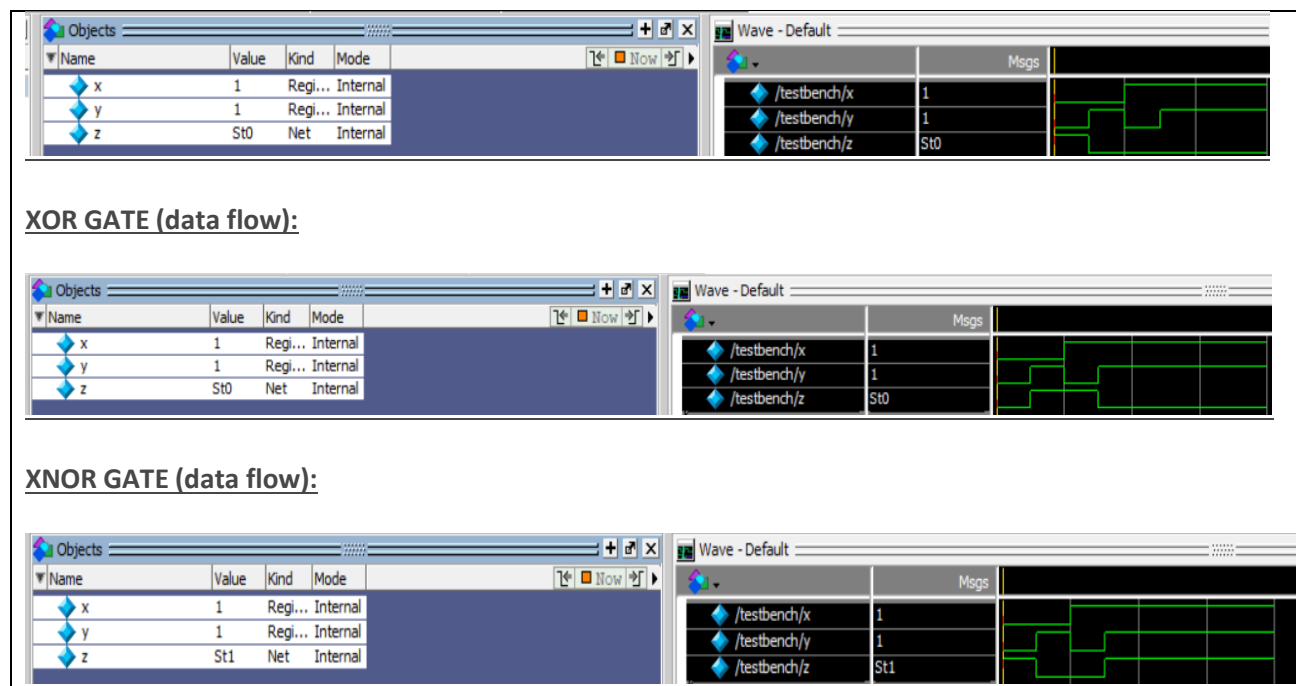
NOT GATE (data flow):



NAND GATE (data flow):



NOR GATE (data flow):



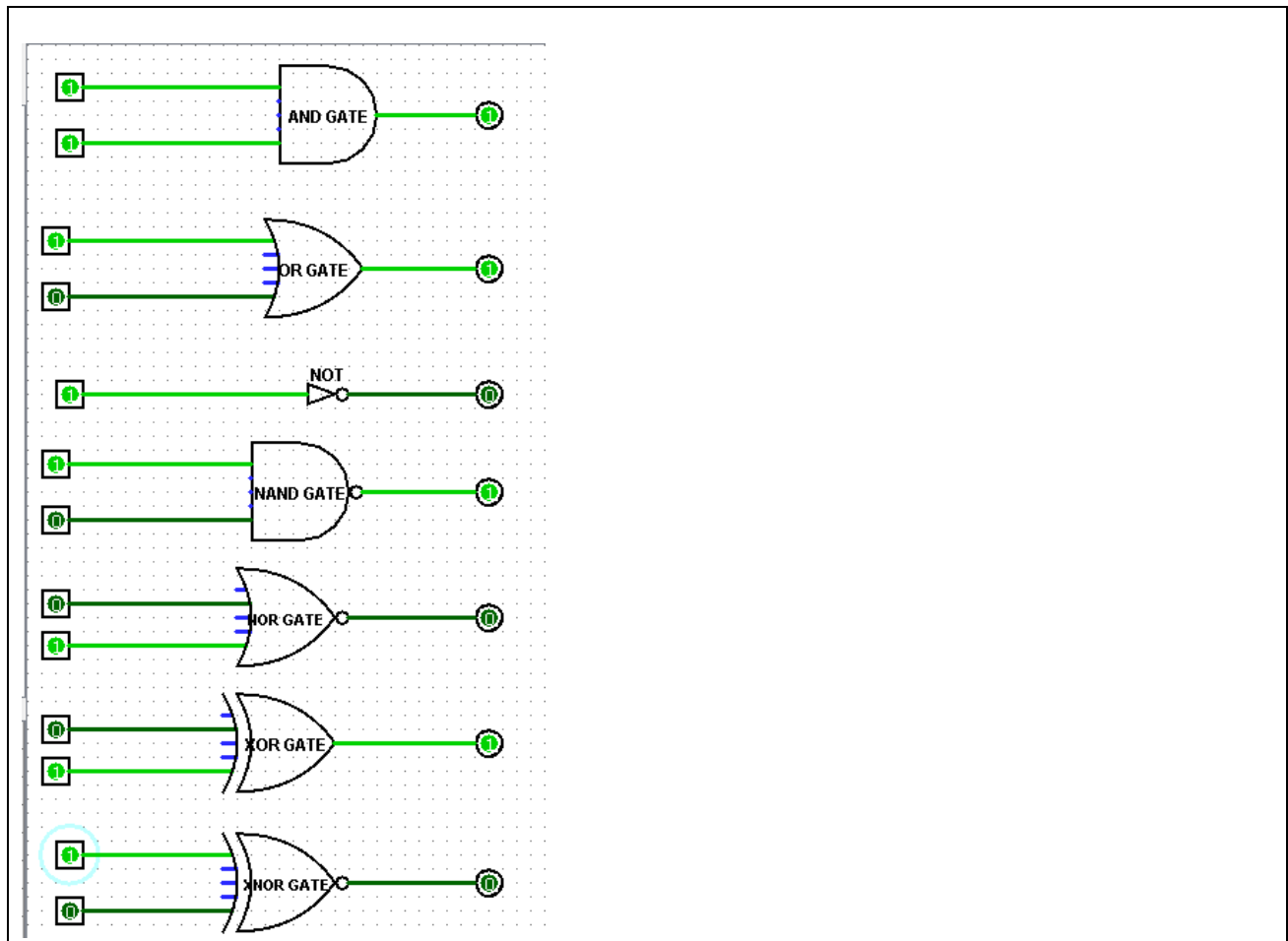
Task 3:

Description:

In the task after writing Verilog codes by both gate level and data flow level modelling the results are verified on Logisim where we make the connections of different basic gates by connecting the inputs and outputs after that checking it with the truth table and the wave that is made on the graph of modelsim. We check it on different combinations of 0s and 1s and compare the results hence they both matched and are correct.

Output:

LOGISIM:



Task 4:

Description:

In this task the circuit diagram is given we have to do it by both gate level and dataflow level. By following the syntax of both modelling this task is performed and here are more gates in the given diagram that are connected with each other so we use more wires(outputs) such as t1, t2, t3, t4 that are representing the output of inner gates and the final output is represented by C so we have to do it by both syntax. The final output is equal to the Xor gate. Here the test bench is also same.

Code:

BY GATE LEVELLING:

```
module LogicGates(a,b,c);
```

```
input a;
```

```
input b;
```

```
output c;

wire t1;

wire t2;

wire t3;

wire t4;

not n1(t1,a);

and a1(t3, t1,b);

not n2(t2,b);

and a2(t4,a,t2);

or o1(c,t3,t4);

endmodule

module testbench();

reg x;

reg y;

wire z;

LogicGates unit(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

end

endmodule
```

BY DATAFLOW LEVELLING:

```
module LogicGates(a,b,c);  
  
input a;  
  
input b;  
  
output c;  
  
wire t1;  
  
wire t2;  
  
wire t3;  
  
wire t4;  
  
assign t1=~a;  
  
assign t3=(t1&b);  
  
assign t2=~b;  
  
assign t4=(a&t2);  
  
assign c=(t3|t4);  
  
endmodule  
  
module testbench();  
  
reg x;  
  
reg y;  
  
wire z;  
  
LogicGates unit(x,y,z);  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50 x=0; y=1;  
  
#50 x=1; y=0;
```

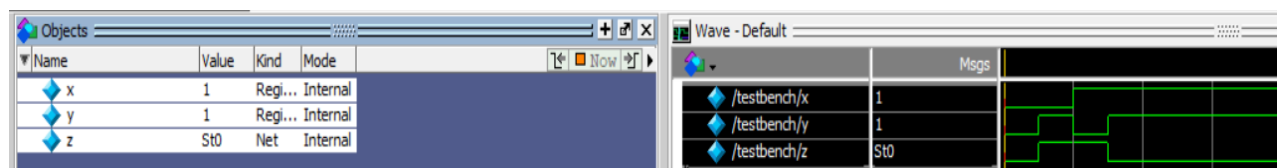
```
#50 x=1; y=1;
```

```
end
```

```
endmodule
```

Output:

BY GATE LEVELLING:



BY DATAFLOW LEVELLING:

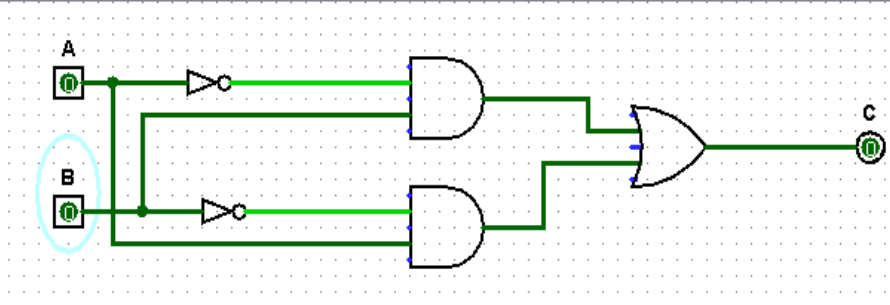


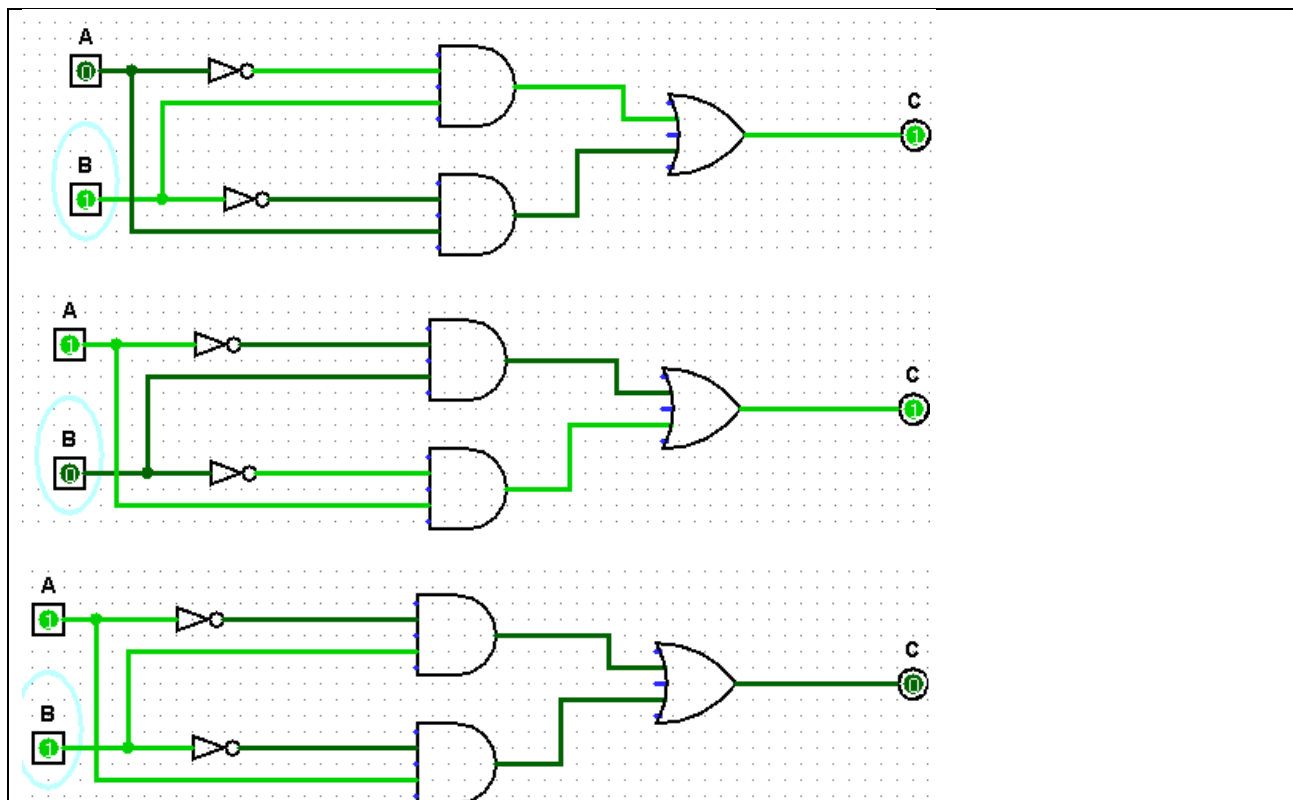
Task 5:

Description:

In this task we are verifying the above task wave by making the circuit simulation on Logisim. Here we check the different combinations of the truth table and matched it with the above wave hence the result is correct.

Output:





Task 6:

Description:

In this task we are assigned to make a 4bit 1's Complement in which the bits are inverted so we make $2^4=16$ combinations are formed. Here we need 4 inputs and 4 outputs and in test bench we also make 4 reg and 4 wires and different 16 combinations to test. We simply use NOT gate to invert or take 1's complement then runs the simulation and the results are shown in the wave. this Verilog code is done by both modelling as required by following their syntax. the results are same by both methods.

Code:

By gate levelling:

```
module LogicGates(a3,a2,a1,a0,b3,b2,b1,b0);

input a0;

input a1;

input a2;

input a3;
```

```
output b0;

output b1;

output b2;

output b3;

not n1(b0,a0);

not n2(b1,a1);

not n3(b2,a2);

not n4(b3,a3);

endmodule

module testbench();

reg A0;

reg A1;

reg A2;

reg A3;

wire B0;

wire B1;

wire B2;

wire B3;

LogicGates unit(A0,A1,A2,A3,B0,B1,B2,B3);

initial

begin

A0=0;A1=0;A2=0;A3=0;

#50 A0=0;A1=0;A2=0;A3=1;

#50 A0=0;A1=0;A2=1;A3=0;
```

```
#50 A0=0;A1=0;A2=1;A3=1;

#50 A0=0;A1=1;A2=0;A3=0;

#50 A0=0;A1=1;A2=0;A3=1;

#50 A0=0;A1=1;A2=1;A3=0;

#50 A0=0;A1=1;A2=1;A3=1;

#50 A0=1;A1=0;A2=0;A3=0;

#50 A0=1;A1=0;A2=0;A3=1;

#50 A0=1;A1=0;A2=1;A3=0;

#50 A0=1;A1=0;A2=1;A3=1;

#50 A0=1;A1=1;A2=0;A3=0;

#50 A0=1;A1=1;A2=0;A3=1;

#50 A0=1;A1=1;A2=1;A3=0;

#50 A0=1;A1=1;A2=1;A3=1;

end

endmodule
```

By dataflow levelling:

```
module LogicGates(a3,a2,a1,a0,b3,b2,b1,b0);

input a0;

input a1;

input a2;

input a3;

output b0;

output b1;

output b2;
```

```
output b3;

assign b0=~a0;

assign b1=~a1;

assign b2=~a2;

assign b3=~a3;

endmodule

module testbench();

reg A0;

reg A1;

reg A2;

reg A3;

wire B0;

wire B1;

wire B2;

wire B3;

LogicGates unit(A0,A1,A2,A3,B0,B1,B2,B3);

initial

begin

A0=0;A1=0;A2=0;A3=0;

#50 A0=0;A1=0;A2=0;A3=1;

#50 A0=0;A1=0;A2=1;A3=0;

#50 A0=0;A1=0;A2=1;A3=1;

#50 A0=0;A1=1;A2=0;A3=0;

#50 A0=0;A1=1;A2=0;A3=1;
```

```
#50 A0=0;A1=1;A2=1;A3=0;

#50 A0=0;A1=1;A2=1;A3=1;

#50 A0=1;A1=0;A2=0;A3=0;

#50 A0=1;A1=0;A2=0;A3=1;

#50 A0=1;A1=0;A2=1;A3=0;

#50 A0=1;A1=0;A2=1;A3=1;

#50 A0=1;A1=1;A2=0;A3=0;

#50 A0=1;A1=1;A2=0;A3=1;

#50 A0=1;A1=1;A2=1;A3=0;

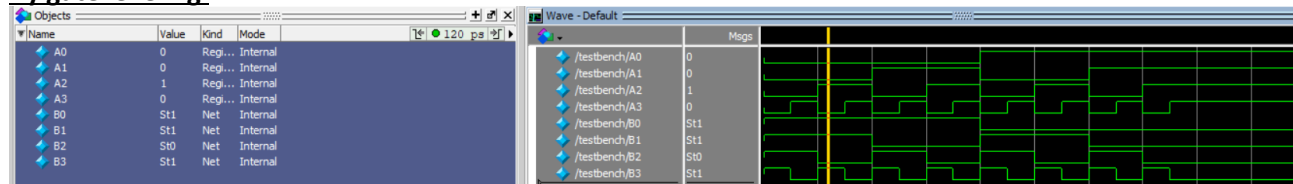
#50 A0=1;A1=1;A2=1;A3=1;

end

endmodule
```

Output:

By gate leveling:



By dataflow levelling:

